Instituto Politécnico Nacional

Escom

Unidad De Aprendizaje

Internet de las cosas / Embedded Systems

Makefiles

Alumna: Kendra Lizeth Torres Gómez

Materia: Internet de las cosas / Embedded Systems

Boleta: 2024630095

Grupo: 6CM3

Fecha de entrega: 25 de abril de 2026

Contenido

1. **Objetivo** 3
2. **Marco Teórico** 3
   a. **¿Qué es Makefile?** 3
   b. **Funcionamiento** 3
   c. **Estructura general** 3
   d. **Elementos importantes** 3
   e. **Objetivos ficticios** 4
   f. **Variables en Makefile** 4
   g. **Reglas patrón** 4
   h. **Ventajas** 4
   i. **Aplicaciones** 4
3. **Desarrollo** 5
   a. **Clonación del repositorio** 5
      i. **ejemplo-basico** 7
      ii. **Julia** 10
   b. **Implementación del proyecto propio** 13
4. **Bibliografía** 16
5. **Repositorio Git** 16

1. Objetivo

Comprender el funcionamiento de los Makefiles como herramienta de automatización en el desarrollo de software, mediante el análisis de ejemplos prácticos y la implementación de un proyecto propio.

2. Marco Teórico

a. ¿Qué es Makefile?

Un Makefile es un archivo de texto que contiene un conjunto de instrucciones utilizadas por la herramienta make para automatizar tareas dentro del desarrollo de software, principalmente la compilación de programas. El Makefile nos permite definir reglas que indican cómo generar un archivo a partir de otros, facilitando la ejecución de múltiples comandos mediante una sola instrucción.

b. Funcionamiento

El funcionamiento de un Makefile se basa en la definición de reglas compuestas por objetivos, dependencias y comandos. Cuando se ejecuta el comando make, el sistema revisa las reglas definidas en el Makefile y determina qué acciones deben ejecutarse en función de las dependencias. Si un archivo ha sido modificado, make recompila únicamente las partes necesarias, evitando procesos redundantes. Además, los Makefiles pueden utilizarse no solo para compilar programas, sino también para ejecutar scripts, limpiar archivos generados o automatizar cualquier tipo de tarea.

c. Estructura general

target: dependencias

comando

Donde:

i. **target (objetivo):** Es el resultado que se desea obtener, como un archivo ejecutable o una acción.

ii. **dependencias:** Son los archivos necesarios para generar el objetivo.

iii. **comando:** Es la instrucción que se ejecuta para crear el objetivo.

d. Elementos importantes

i. **Objetivos**

Representan el resultado final o una acción a ejecutar. Pueden ser archivos o simplemente nombres de tareas.

ii. **Dependencias**

Son los archivos que necesita un objetivo para generarse. Si estos cambian, el objetivo se vuelve a ejecutar.

iii. **Comandos**

Son las instrucciones que se ejecutan en la terminal para cumplir un objetivo.

e. **Objetivos ficticios**

Los objetivos ficticios no representan archivos reales, sino acciones. Se definen con la palabra clave .PHONY para evitar conflictos con archivos del mismo nombre. Por ejemplo:

i. **.PHONY: clean**

Esto asegura que el comando se ejecute siempre, independientemente de si existe un archivo llamado "clean".

f. **Variables en Makefile**

Los Makefiles permiten definir variables para evitar repetir código y facilitar la modificación de valores. Por ejemplo:

i. **CC = gcc**

Esto permite reutilizar el compilador en diferentes partes del archivo.

g. **Reglas patrón**

Las reglas patrón permiten generalizar procesos, como convertir automáticamente archivos .c en .o. Por ejemplo:

i. **%.o: %.c**

Esto indica que cualquier archivo .c puede transformarse en un archivo .o.

h. **Ventajas**

i. Automatización de tareas repetitivas
ii. Reducción de errores humanos
iii. Optimización del tiempo de compilación
iv. Organización del proyecto
v. Facilidad para trabajar en equipo

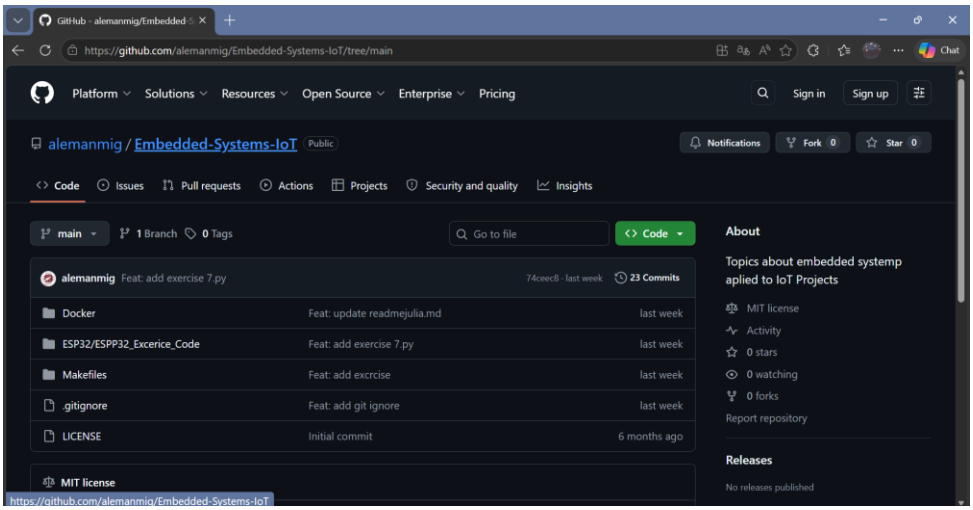
i. **Aplicaciones**

i. Compilación de programas en C/C++
ii. Ejecución de scripts
iii. Automatización de pruebas
iv. Limpieza de archivos temporales
v. Construcción de proyectos complejos

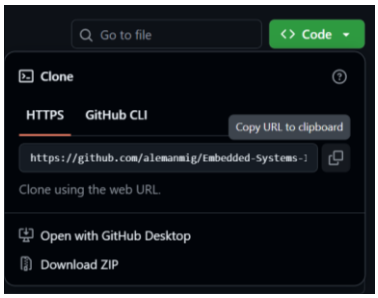
3. Desarrollo

a. Clonación del repositorio

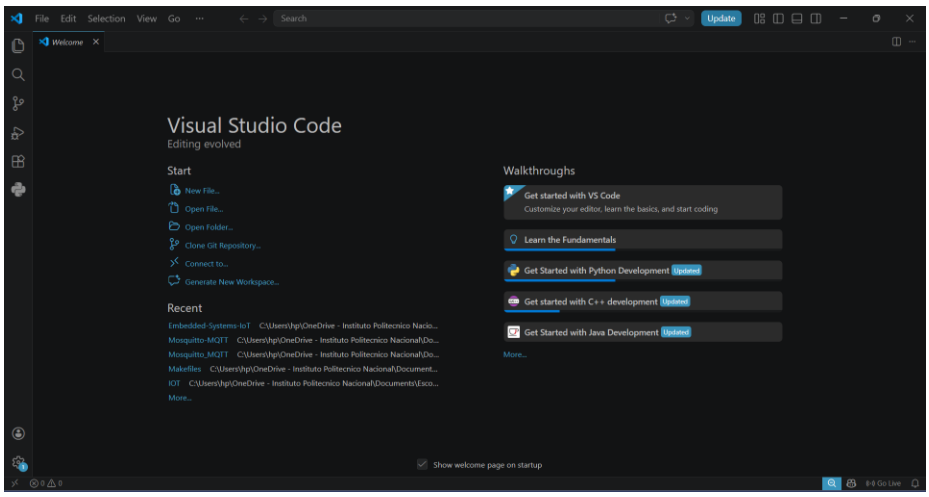
Para realizar la clonación del repositorio, primero se ingresa al link y nos vamos a la carpeta raíz:



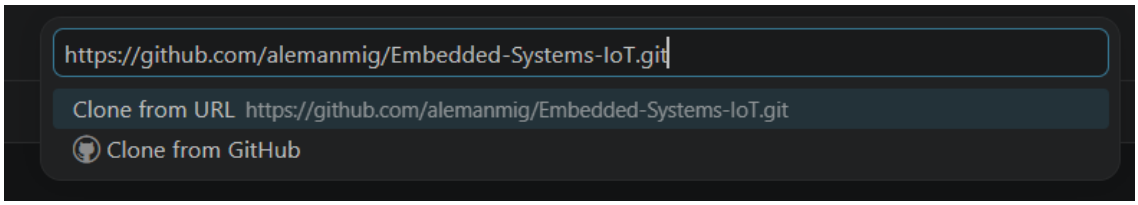
Después le damos clic donde dice "code" y nos va a desplegar una pestaña donde nos da una URL del repositorio:



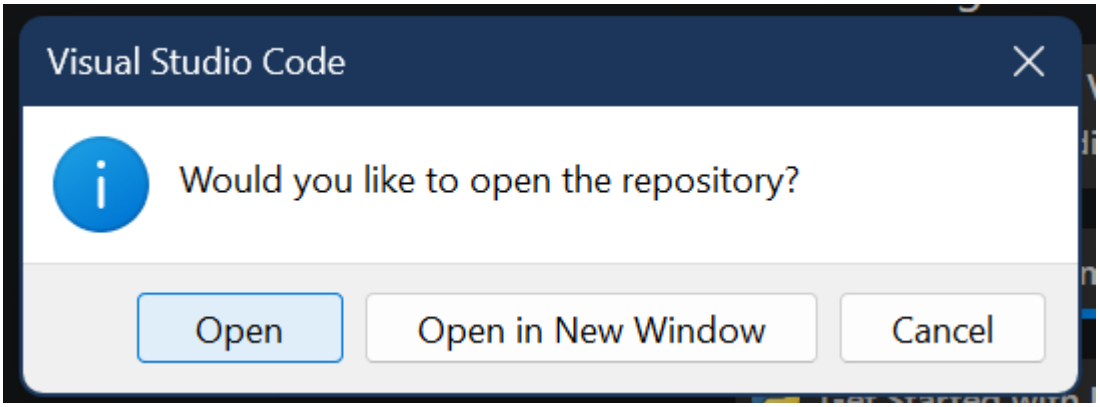
Copiamos la URL. Después, abrimos Visual Studio y en la pantalla de inicio buscamos "Clone Git Repository":



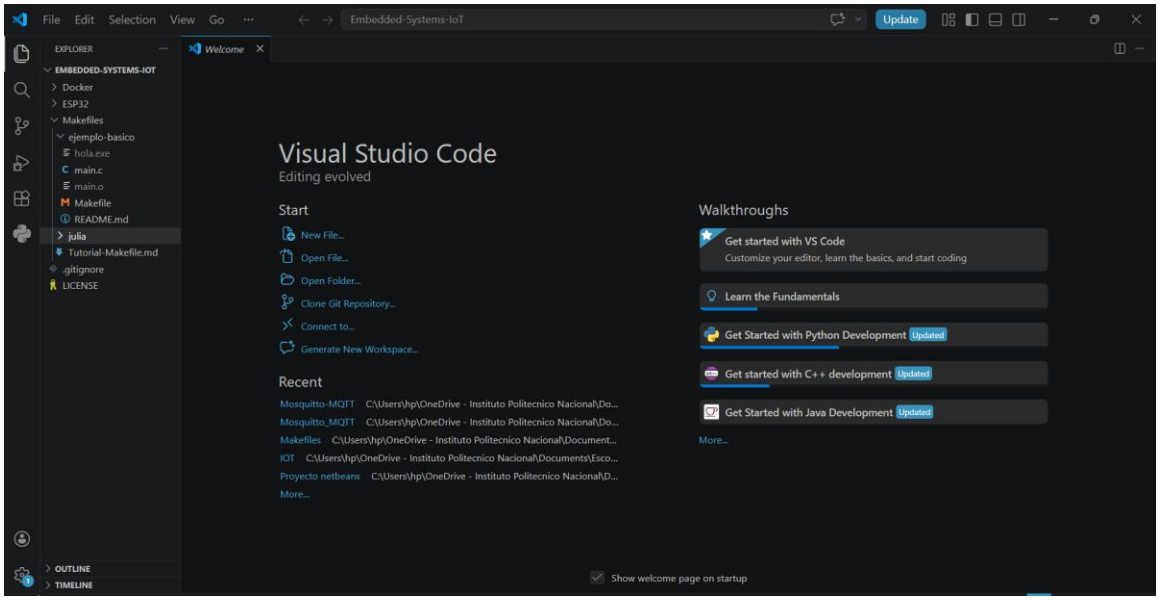
Le damos click y nos va a desplegar un pestaña de búsqueda donde vamos a copiar el link y posteriormente le damos click en “Clone from URL”:



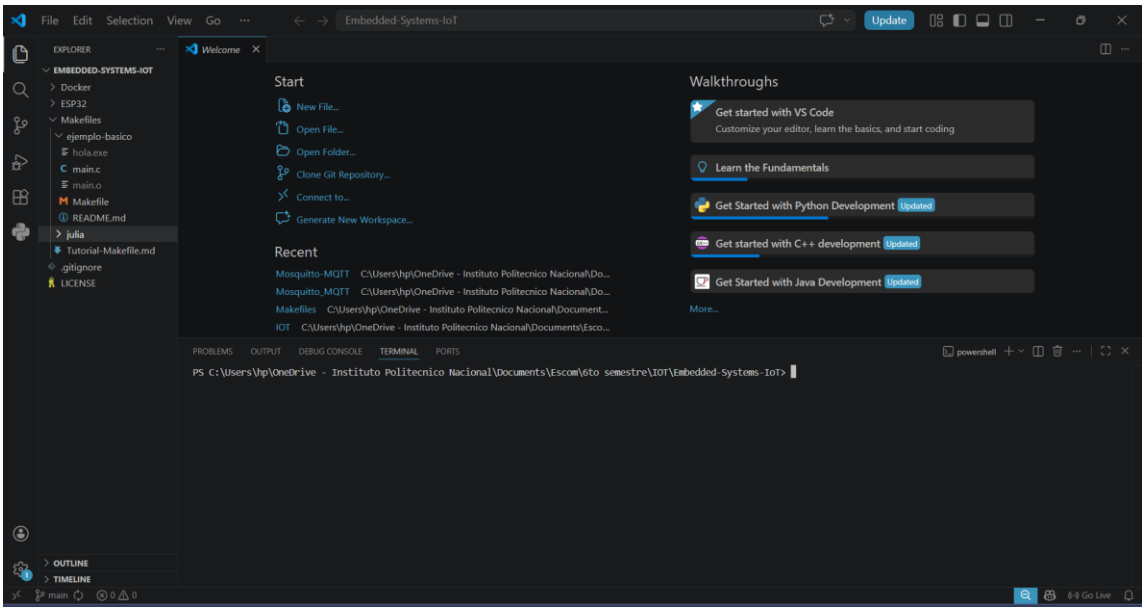
Nos va a preguntar donde deseamos guardarlo y después nos va a preguntar si deseamos abrirlo, por lo tanto, le damos click en “open”:



Una vez hecho esto, podremos ver las carpetas en el lado izquierdo de nuestro VS.



En este caso la carpeta a usar será “Makefiles” donde podemos encontrar los dos ejemplos a analizar que son “ejemplo-basico” y “julia”. Por lo tanto, antes de iniciar se deberá abrir una nueva terminal con “ctrl + ñ”.



i. ejemplo-basico

1. main.c

Contiene un programa en lenguaje C que imprime mensajes en pantalla para demostrar la ejecución del programa.

```c
#include <stdio.h>

int main(void) {
    printf("Hola desde un ejemplo basico de Makefile.\n");
    printf("Este programa fue compilado y ejecutado con make.\n");
    return 0;
}
```

2. Makefile

El Makefile define las reglas necesarias para compilar, ejecutar y limpiar el proyecto de forma automatizada.

a. Compilación del código fuente
b. Generación del ejecutable
c. Ejecución
d. Limpieza de archivos

Sus components son:

a. Variables

i. CC = gcc

Define el compilador que se utilizará.

ii. CFLAGS = -Wall -Wextra -O2

Son opciones de compilación:

1. -Wall activa advertencias
2. -Wextra activa advertencias adicionales
3. -O2 optimiza el código

iii. **TARGET = hola**

Nombre del archivo ejecutable final.

iv. **SRCS = main.c**

Lista de archivos fuente.

v. **OBJS = $(SRCS:.c=.o)**

Convierte automáticamente archivos .c en .o (archivos objeto).

b. **Objetivos especiales**

i. **.PHONY: all run clean help rebuild**

Indica que estos objetivos no son archivos reales, sino comandos.

c. **Objetivo principal**

i. **all: $(TARGET)**

Este es el objetivo por defecto. Al ejecutar make, se compila el programa.

d. **Ayuda**

i. **help: @echo "Objetivos disponibles:"**

Muestra información en la terminal sobre los comandos disponibles.

e. **Regla patrón**

i. **%.o: %.c $(CC) $(CFLAGS) -c $< -o $@**

Esta regla indica cómo convertir archivos .c en .o

1. **$<** representa el archivo fuente
2. **$@** representa el archivo destino

f. **Construcción del ejecutable**

i. **$(TARGET): $(OBJS) $(CC) $(CFLAGS) $(OBJS) -o $@**

Indica que el ejecutable depende de los archivos objeto.

g. **Ejecutar el programa**

i. **run: $(TARGET) ./$(TARGET)**

Compila si es necesario y ejecuta el programa.

h. **Limpiar archivos**

i. **clean: rm -f $(OBJS) $(TARGET)**

Elimina archivos generados (.o y ejecutable).

i. **Reconstrucción**

i. **rebuild: clean all**

Primero limpia y luego vuelve a compilar todo.

j. **Flujo de automatización**

i. **make:** compila

ii. **make run:** compila y ejecuta

iii. **make clean:** limpia

iv. **make rebuild:** recompila desde cero

3. **Ejecución**

Una vez teniendo esto en cuenta, se ingresó a la carpeta correspondiente del ejemplo básico mediante la terminal:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT> cd Makefiles
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles> cd ejemplo-basico
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles\ejemplo-basico>
```

Ya que nos encontramos en la carpeta correspondiente, ejecutamos el comando "make":

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles\ejemplo-basico> make
gcc -Wall -Wextra -O2 -c main.c -o main.o
gcc -Wall -Wextra -O2 main.o -o hola
```

Este comando utilizó el Makefile para automatizar el proceso de compilación del programa. Internamente, se invocó el compilador gcc, el cual realizó dos acciones principales: primero compiló el archivo fuente main.c generando un archivo objeto (main.o), y después enlazó dicho archivo para producir el ejecutable final llamado hola. Una vez generado el ejecutable, se procedió a ejecutar el programa con el comando "./hola".

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles\ejemplo-basico> ./hola
Hola desde un ejemplo basico de Makefile.
Este programa fue compilado y ejecutado con make.
```

Al ejecutar el programa, se ven en pantalla los mensajes definidos en el código fuente, comprobando que la compilación y ejecución se realizaron correctamente mediante el uso del Makefile.

ii. Julia

Este programa lo que hace es generar una imagen del conjunto de Julia que es un fractal. El flujo que sigue es que con main.cpp genera datos numéricos en un txt, después el gnuplot convierte esos datos en una imagen png y el Makefile automatiza todo.

1. main.cpp

Es el programa principal en C++ el cual calcula los valores matemáticos del fractal, después usa esos números complejos y genera un archivo llamado "julia_set.txt" el cual contiene las coordenadas "x,y" y el valor del fractal. Por lo tanto, es la base para generar la imagen.

```cpp
#include <cmath>
#include <complex>
#include <iomanip>
#include <iostream>
#include <fstream>

double mandelbrot (const double& real, const double& imag) {

  // (-0.70176) (-0.3842)
  // (-0.835)  (-0.2321)
  // (0.4)  (-0.325)
  double z_real = real;
  double z_imag = imag;

  int iter = 45;
  double max_iter = static_cast<double>(iter);

  double zn_real; double zn_imag;
  for (int n = 0; n < iter; n++) {

    zn_real = (z_real * z_real) - (z_imag * z_imag) + (-0.70176);
    zn_imag = 2.0 * z_real * z_imag + (-0.3842);

    if ( zn_real*zn_real + zn_imag*zn_imag > 4.0 ) {
      return static_cast<double>(n);
    }

    z_real = zn_real;
    z_imag = zn_imag;

  }

  return max_iter;
}
```

```cpp
int main(int argc, char** argv) {
  double x0 = -0.0;
  double r = 1.8;
  double points = 500;

  std::string filename = "julia_set.txt";
  std::ofstream outfile;
  outfile.open(filename);

  // Set fixed-point notation and precision
  outfile << std::fixed << std::setprecision(5);

  for (double y = -r; y < r; y += (r+r)/(points-1.0)) {
    for (double x = x0-r; x < x0+r; x += (r+r)/(points-1.0)) {
      outfile << std::setw(8) << x << ", "
              << std::setw(8) << y << ", "
              << std::setw(8) <<  mandelbrot(x,y) <<
              std::endl;

    }
    outfile << std::endl;
  }

  outfile.close();

  return 0;
}
```

2. julia_set.txt

Es el archivo generado por el programa que sirve como entrada para graficar.

```
-1.80000, -1.80000,  0.00000
-1.79279, -1.80000,  0.00000
-1.78557, -1.80000,  0.00000
-1.77836, -1.80000,  0.00000
-1.77114, -1.80000,  0.00000
```

3. julia_set.gp

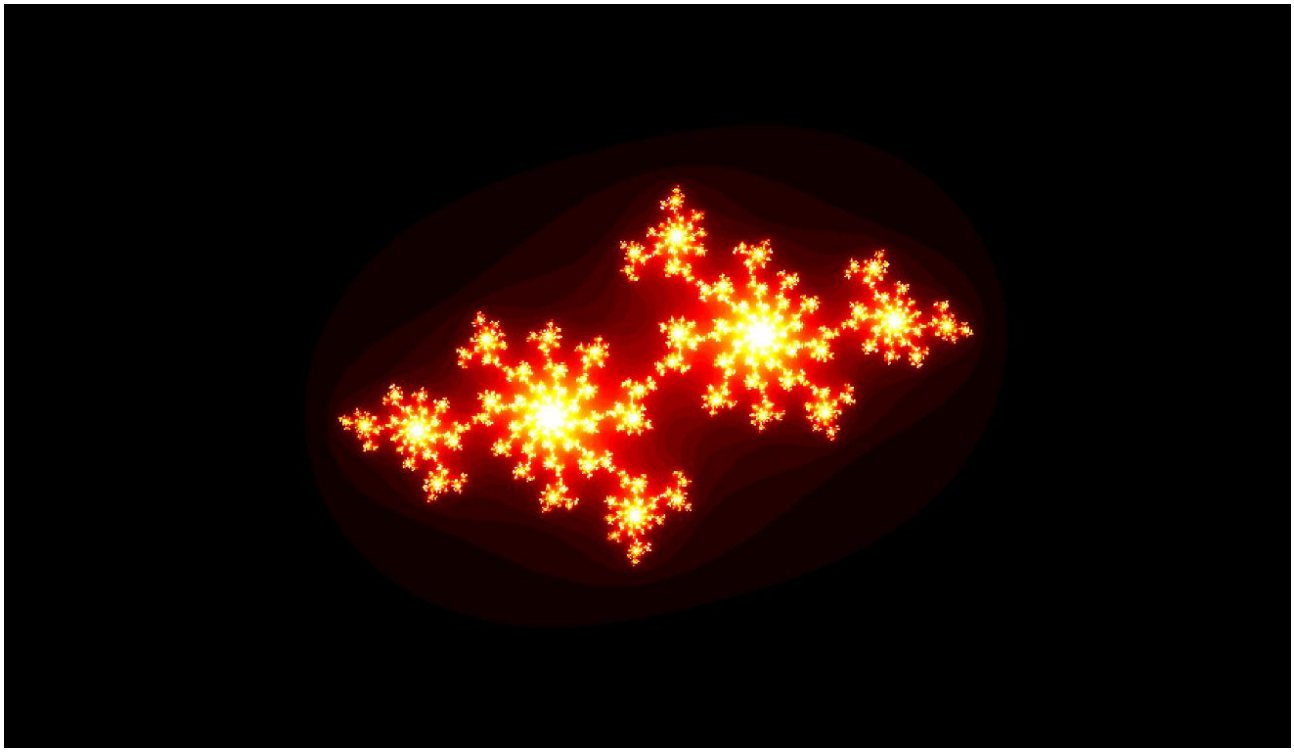
Es el script de gnuplot que lee el archivo .txt y genera una imagen. Este puede producir tanto png, pdf y LaTex, y configura colores, tamaños y formatos. Es el que convierte los datos en imagen.

```gp
# Author: Ciro Fabian Bermudez Marquez
# File: julia_set.gp
# Description: julia_set
# Run: gnuplot julia_set.gp

filename = "julia_set"
file = filename . ".txt"
select = 1

# Change the terminal from the command line
if (ARGC > 0) {
    select = ARG1 + 0
}

if (select == 1) {
# PNG Version
  print "PNG Mode Selected"
  reset
  unit = 3
  # use crop
  set terminal pngcairo size unit*640, unit*384 font "CMU Serif, 10" fontscale 1*unit*0.95 background rgb "□#000000"
  set output filename . ".png"
}
if (select == 2) {
# PDF Version
  print "PDF Mode Selected"
  reset
  unit = 3
  set terminal pdfcairo size unit*5in, unit*3in font "CMU Serif, 10" fontscale 0.5*unit background rgb "□#000000"
  set output filename . ".pdf"
}
if (select == 3) {
# LaTeX Version
  print "LaTeX Mode Selected"
  reset
  unit = 3
  set terminal epslatex standalone color size unit*5in, unit*3in background rgb "□#000000"
  set output filename . ".tex"
}
```

```gp
# Disable x-axis
unset xtics
unset xlabel

# Disable y-axis
unset ytics
unset ylabel

# Disable the border completely
set border 0

# Disable legend
unset key

#set palette defined ( 0 'blue', 45 'yellow')
#set palette rgbformula -7,2,-7
#set palette rgb 7,5,15;


# Set color palette
set palette rgb 21,22,23;
set size square
unset colorbox

plot file using 1:2:3 with image \
    #,file using 1:2:(sprintf("%g",$3) ) with labels

# Restore default values
reset
set terminal qt
set output
```

4. julia_set.png

Es el resultado final, es la imagen del fractal y es generada automáticamente.



5. Makefile

Automatiza todo el proceso y lo que hace es que:

a. Compila (main.cpp)

b. Ejecuta programa y genera .txt

c. Ejecuta gnuplot y genera .png

d. Abre la imagen

Usa targets importantes que son:

e. all que hace todo

f. run que ejecuta programa

g. plot que genera imagen

h. open que abre imagen

i. clean que limpia archivos

```makefile
# Variables
CXX = g++
CXXFLAGS = -std=c++23 -O3
GP = julia_set.gp
TXT = $(GP:.gp=.txt)
PNG = $(GP:.gp=.png)
SRCS = main.cpp
OBJS = $(SRCS:.cpp=.o)
APP  = julia

## TARGETS
all: run plot open
.PHONY: vars
vars:
    @echo "print variables"
    @echo "SRCS = $(SRCS)"
    @echo "OBJS = $(OBJS)"
    @echo "PRE  = $<"
    @echo "NAME = $@"
    @echo "GP   = $(GP)"
    @echo "TXT  = $(TXT)"
%.o: %.cpp
    @echo "Compiling"
    $(CXX) $(CXXFLAGS) -c $< -o $@
$(APP): $(OBJS)
    @echo "Linking"
    $(CXX) $(CXXFLAGS) $(OBJS) -o $(APP)
run: $(APP)
    @echo "RUN"
    ./$(APP)
plot: $(TXT)
    @echo "Plot"
    gnuplot $(GP)
open:
    @echo "Open"
    xdg-open $(PNG) &
clean:
    rm *.o $(APP) *.txt *.png
```

6. **Makefile.old**

   Básicamente es la versión mas avanzada que detecta el sistema operativo, permite elegir el formato ya sea png o pdf e incluye el formateador de código. Aunque no es realmente necesario usarlo.

7. **test.gp**

   Es el script de prueba que genera el grafico en 3D y sirve para experimentar con gnuplot. Aunque no se usa.

8. **Ejecución**

   Ingresamos a la carpeta correspondiente:

   Ejecutamos el comando "make":

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT> cd Makefiles
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles> cd julia
```

   Una vez estando dentro de la carpeta, ejecutamos "make julia".

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles\julia> make julia
make: `julia' está actualizado.
```

   Y por último, ejecutamos el programa para generar la imagen julia_set.png:

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles\julia> ./julia
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Embedded-Systems-IoT\Makefiles\julia> gnuplot julia_set.gp
PNG Mode Selected
```

b. Implementación del proyecto propio

Para esto se desarrolló un código que genera una imagen.bmp con ondas usando matemáticas y usa un Makefile para automatizar la compilación, la ejecución y la limpieza.

i. Código de ondas.c

```c
#include <stdio.h>
#include <math.h>
#include <stdint.h>

#define WIDTH 800
#define HEIGHT 800

// Estructura para la cabecera del BMP
#pragma pack(push, 1)
typedef struct {
    uint16_t type;
    uint32_t size;
    uint16_t reserved1, reserved2;
    uint32_t offset;
    uint32_t header_size;
    int32_t  width, height;
    uint16_t planes;
    uint16_t bpp;
    uint32_t compression;
    uint32_t img_size;
    int32_t  res_x, res_y;
    uint32_t colors, important_colors;
} BMPHeader;
#pragma pack(pop)
```

```c
int main() {
    FILE *fp = fopen("ondas.bmp", "wb");
    BMPHeader header = {0};
    header.type = 0x4D42;
    header.offset = sizeof(BMPHeader);
    header.header_size = 40;
    header.width = WIDTH;
    header.height = HEIGHT;
    header.planes = 1;
    header.bpp = 24;
    header.size = header.offset + (WIDTH * HEIGHT * 3);

    fwrite(&header, sizeof(header), 1, fp);

    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            float d = sqrt(pow(x - 400, 2) + pow(y - 400, 2));

            unsigned char b = (unsigned char)(127 + 127 * sin(d * 0.1));
            unsigned char g = (unsigned char)(127 + 127 * cos(d * 0.05));
            unsigned char r = (unsigned char)(255 - (b + g) / 2);

            fputc(b, fp);
            fputc(g, fp);
            fputc(r, fp);
        }
    }

    fclose(fp);
    printf("Imagen 'ondas.bmp' generada exitosamente.\n");
    return 0;
}
```

Básicamente lo que hace el código es que primero define las dimensiones de la imagen con WIDTH y HEIGHT, ambas de 800 píxeles. Después, define una estructura BMPHeader, la cual representa la cabecera del archivo BMP. Esta estructura tiene la información como el tamaño de la imagen, dimensiones, tipo de archivo y formato de color. Posteriormente, en el main se crea un archivo llamado ondas.bmp en modo binario. Luego, se inicializa la cabecera BMP con los valores necesarios y se escribe en el archivo. Después, se recorren todos los píxeles de la imagen con dos ciclos anidados para las coordenadas (x,y). Para cada píxel, se calcula la distancia respecto al centro de la imagen y se aplican funciones seno y coseno para generar valores de color. Los valores calculados se asignan a los componentes RGB (rojo, verde y azul), lo que crea un patrón visual de ondas. Por último, los valores de color se escriben en el archivo y se cierra el archivo, generando la imagen completa.

ii. Makefile

El Makefile, en este proyecto, tiene como objetivo automatizar la compilación, ejecución y limpieza del programa ondas.c, el cual genera una imagen en formato BMP.

```
CC = gcc
CFLAGS = -Wall
LIBS = -lm
TARGET = simulacion

all: $(TARGET)

$(TARGET): ondas.c
	$(CC) $(CFLAGS) ondas.c -o $(TARGET) $(LIBS)

run: $(TARGET)
	./$(TARGET)

clean:
	rm -f $(TARGET) Ondas.ppm
```

1. Variables

a. CC = gcc que define el compilador que se utilizará para construir el programa.

b. CFLAGS = -Wall que activa advertencias durante la compilación, lo que ayuda a detectar posibles errores.
c. LIBS = -lm que indica que se debe enlazar la librería matemática, necesaria para funciones como sin, cos y sqrt.
d. TARGET = simulación que especifica el nombre del archivo ejecutable que se generará.

2. **Objetivo principal**
   a. all: $(TARGET) define el objetivo por defecto del Makefile. Al ejecutar make, se construye automáticamente el programa.
3. **Regla de compilación**
   a. $(TARGET): ondas.c que ndica que el ejecutable depende del archivo fuente ondas.c.
   b. $(CC) $(CFLAGS) ondas.c -o $(TARGET) $(LIBS) el cual es el comando que compila el archivo ondas.c y genera el ejecutable llamado simulacion.
4. **Ejecución del programa**
   a. run: $(TARGET) define un objetivo que depende del ejecutable.
   b. ./$(TARGET) ejecuta el programa generado.
   De esta forma, al usar make run, el programa se compila (si es necesario) y se ejecuta automáticamente.
5. **Limpieza de archivos**
   a. clean: define un objetivo para eliminar archivos generados.
   b. rm -f $(TARGET) Ondas.ppm elimina el ejecutable y archivos generados, permitiendo mantener limpio el proyecto.
6. **Automatización**
   Este Makefile permite automatizar todo el flujo de trabajo del programa:
   a. make compila el programa
   b. make run compila y ejecuta
   c. make clean elimina archivos generados

iii. Ejecución

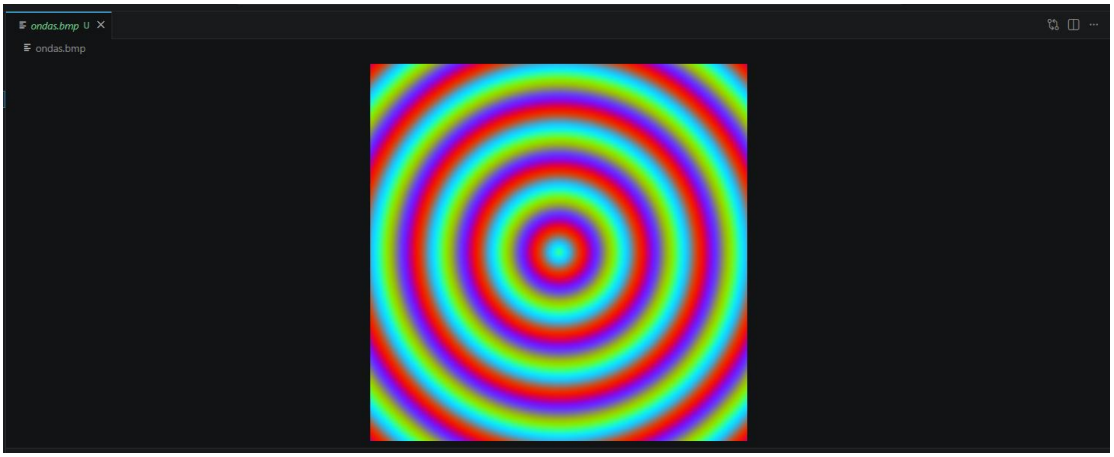
Una vez que el repositorio fue clonado, abrimos la terminal y ejecutamos el comando “make”.

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Makefiles> make
gcc -Wall ondas.c -o simulacion -lm
```

Después, ejecutamos el comando “make run”.

```
PS C:\Users\hp\OneDrive - Instituto Politecnico Nacional\Documents\Escom\6to semestre\IOT\Makefiles> make run
gcc -Wall ondas.c -o simulacion -lm
./simulacion
Imagen 'ondas.bmp' generada exitosamente.
```

Una vez realizado estos pasos, podremos ver el resultado el cual es una imagen.bmp con ondas.



Podemos concluir que el Makefile desarrollado permite automatizar completamente el flujo de trabajo del programa. En lugar de ejecutar múltiples comandos manualmente, como la compilación con gcc y la ejecución del programa, el usuario solo necesita ejecutar make o make run. Esto reduce errores, mejora la eficiencia y estandariza el proceso de desarrollo, ya que cualquier usuario puede ejecutar el proyecto siguiendo los mismos pasos definidos en el Makefile.

4. Bibliografía

a. https://www.gnu.org/software/make/manual/make.pdf

b. https://opensource.com/article/18/8/what-how-makefile

5. Repositorio Git

a. https://github.com/Liz12tg/Makefiles.git